Homework 13

Xander Jhaveri

Screenshots Showing `PORTA` Stimulus

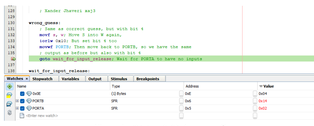

Figure 1: `PORTA = 0x02`, Incorrect Guess

Figure 2: `PORTA = 0x02`, Correct Guess

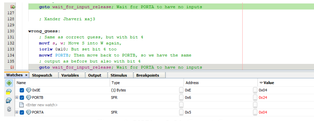

Figure 3: `PORTA = 0x04`, Correct Guess

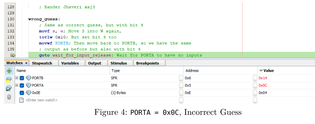

Figure 4: `PORTA = 0x0C`, Incorrect Guess

Figure 5: `PORTA = 0x00, S = 0x01`

Figure 6: `PORTA = 0x00, S = 0x02`

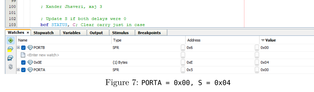

Figure 7: `PORTA = 0x00, S = 0x04`

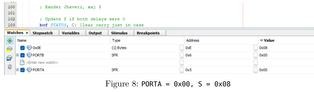

Figure 8: `PORTA = 0x00, S = 0x08`

homework13.asm

```
; Xander Jhaveri - xaj3

    list      p=16F84A              ; list directive to define processor
    #include <p16F84a.inc>          ; processor specific variable definitions

    __CONFIG   _CP_OFF & _WDT_OFF & _PWRTE_ON & _LP_OSC

; '__CONFIG' directive is used to embed configuration data within .asm file.
; The lables following the directive are located in the respective .inc file.
; See respective data sheet for additional information on configuration word.

;***** VARIABLE DEFINITIONS
w_temp        EQU     0x0C        ; variable used for context saving
status_temp   EQU     0x0D        ; variable used for context saving

s       EQU     0x0E
delay_ls        EQU     0x0F
delay_ms        EQU     0x10

RESET_VECTOR        CODE    0x0000  ; processor reset vector
        goto    start               ; go to beginning of program

ISR             CODE    0x0004  ; interrupt vector location

Interrupt:
        movwf   w_temp              ; save off current W register contents
        movf    STATUS,w            ; move status register into W register
        movwf   status_temp         ; save off contents of STATUS register

        movf    status_temp,w       ; retrieve copy of STATUS register
        movwf   STATUS              ; restore pre-isr STATUS register contents
        swapf   w_temp,f
        swapf   w_temp,w            ; restore pre-isr W register contents
        retfie                      ; return from interrupt

MAIN_PROGRAM    CODE

; Delay Notes:
    ; At a 100kHz clock frequency, there are 25k instructions processed per
sec
    ; So we need to go through 25000 clock cycles before changing S

    ; Every iteration of the main loop (not including changing S) is about 7
cycles:
    ; - 1 cycle for clrf PORTB
    ; - 1 cycle for the movf,
    ; - 1 cycle for the andlw
    ; - 2 cycle for the btfss, (assuming a skip)
    ; - 1 cycle for the decfsz,
    ; - 2 cycles for the goto main_loop
    ; It could be 9-10 if delay_ls is zero, but approximately 8 is fine

    ; This means that the main loop will need to run 25000 / 8 = 3125 times
    ; Before changing S

    ; I will achieve this using two delay counter variables.
    ; - A least significant counter: delay_ls that counts down from 100
    ;- Upon reaching 0, it will decrement MS counter
    ; - A most significant counter: delay_ms that counts down from 31
    ;- Upon reaching 0, S will change

start:
    ; PORT Config
    bsf STATUS, RP0 ; Switch to Bank 1
    movlw 0x00 ; Move 1100 0000 into W
    movwf TRISB ; Move 1100 0000 into TRISB, TRISA is already all inputs
    bcf STATUS, RP0 ; Switch back to Bank 0

    ; Initialize delay counters
    movlw 0x1F;
    movwf delay_ms; Set most significant counter delay to 31
    movlw 0x64;
    movwf delay_ls; Set least significant counter delay to 100
    ; Initialize S

reset_s:
    movlw 0x01 ; Start with state S1 by moving 1 into state variable
    movwf s ;

main_loop:
    ; Clear port B
    clrf PORTB;
    ; Check for Input in PORTA
    movf PORTA, w;
    andlw 0x0F; Move the least significant 4 bits of PORTA into W
    btfss STATUS, Z; If PORTA has no input, skip
    goto handle_input; Goto handle_input whenever there is input

    ; Decrement counters and check if they are zero
    decfsz delay_ls, f; Decrement ls, if 0, then reset
    goto main_loop

    movlw 0x64;
    movwf delay_ls; Reset ls to 100

    decfsz delay_ms, f; Decrement ms, if 0, then change S
    goto main_loop

    movlw 0x1F;
    movwf delay_ms; Reset ms to 31

    ; Update S if both delays were 0
    bcf STATUS, C; Clear carry just in case
    rlf s, f; Rotate S left
    movf s, w;
    andlw 0x0F; Move the leasy significant 4 bits of S into W
    movwf s; Move only the 4 bits back to S
    btfsc STATUS, Z; If we rotated beyond 1000, then w is 0000, so we reset s
    goto reset_s;
    goto main_loop; Otherwise we just go back to the main loop with reset
delays

handle_input:
    movf PORTA, w;
    andlw 0x0F; Move the least significant 4 bits of PORTA to W
    xorwf s, W; If S and W (PORTA) are equal, then this results in zero

    btfsc STATUS, Z; If result was 0, it was the correct guess
    goto correct_guess;
    goto wrong_guess; Otherwise, wrong guess

correct_guess:
    movf s, w; Move S into W again,
    iorlw 0x20; But set bit 5 too
    movwf PORTB; Then move back to PORTB, so we have the same
    ; output as before but also with bit 5
    goto wait_for_input_release; Wait for PORTA to have no inputs

wrong_guess:
    ; Same as correct guess, but with bit 4
    movf s, w; Move S into W again,
    iorlw 0x10; But set bit 4 too
    movwf PORTB; Then move back to PORTB, so we have the same
    ; output as before but also with bit 4
    goto wait_for_input_release; Wait for PORTA to have no inputs

wait_for_input_release:
    movf PORTA, w; Move the least significant 4 bits of PORTA to W
    andlw 0x0F; If this is 0, then PORTA has no inputs
    btfss STATUS, Z; If not zero, keep waiting
    goto wait_for_input_release
    goto main_loop; If zero, go back to main loop

    END
```